

Отчет по заданию «А4. Сравнительный анализ алгоритмов поиска мажоритарного элемента»

Выполнил: студент Ошаров Александр группы БПИ248

Семинарист: Маров Руслан

Аннотация

В отчёте проводится проверка корректности и оценка временной сложности трёх алгоритмов, предназначенных для поиска мажоритарного элемента (элемента, встречающегося более чем в $n/2$ позициях). Даны примеры, где алгоритмы совпадают и где расходятся результаты, предложены минимальные доработки, гарантирующие корректность при сохранении асимптотики.

1. Уточнения и модель

Замечание по индексам: в тексте алгоритмы формулируются с циклами «for i = 0 to n» или «for i = 1 to n»; в традиционной реализации массив A имеет индексы 0..n-1. В отчёте предполагается, что циклы проходят по всем элементам массива, то есть от 0 до n-1 (или эквивалентно от 1 до n с соответствующей сдвиговой нотацией). Это не влияет на асимптотику и логику, но в трассировках я даю индексы 0..n-1.

2. Краткое описание алгоритмов и их задачи

Задача: найти мажоритарный элемент — элемент x, для которого число вхождений $> n/2$. Если такой элемент существует, он единственен. Корректный алгоритм на входе должен вернуть этот элемент; если мажоритарного элемента нет, корректное поведение — явно сообщить об отсутствии (или не возвращать неверное значение).

2.1. algorithm1 (полный перебор)

Суть: для каждого A[i] считаем, сколько раз он встречается (вложенный цикл); запоминаем максимум; в конце возвращаем A[ind], только если $c > n/2$.

Вывод: корректен: при наличии мажоритарного элемента он будет найден; при его отсутствии алгоритм не вернёт ложный элемент (он возвращает только при $c > n/2$).

2.2. algorithm2 (Бойера–Мура — поиск кандидата)

Суть: за один проход восстанавливается кандидат на мажоритарный элемент (метод Бойера–Мура). В текущей форме алгоритм возвращает найденный кандидат, но не проверяет, является ли он действительно мажоритарным.

Вывод: без финальной проверки алгоритм может возвращать произвольный элемент, когда мажоритарного элемента нет — некорректен в общем случае, но в случае существования мажоритарного элемента он гарантированно вернёт этот элемент как кандидат.

2.3. algorithm3 (сортировка и подсчёт подряд равных элементов)

Суть: сортируем массив, затем проходим и считаем длину текущего блока одинаковых элементов; если длина блока превысит $n/2$ — возвращаем этот элемент. В текущей записи отсутствует финальная проверка после цикла, поэтому последний блок может быть проигнорирован.

Вывод: корректен при добавлении проверки последнего блока; без неё алгоритм может не вернуть мажоритарный элемент, если он расположен в конце отсортированного массива.

3. Корректность — наглядные примеры и трассировки

Пример 1 — массив с мажоритарным элементом:

$A = [3, 3, 3, 2, 1]$, $n = 5$ (мажоритарный элемент 3, встречается 3 раза)

Трассировка (кратко):

algorithm1: для $i=0$ найдёт $c1=3 \rightarrow c=3$, $ind=0 \rightarrow$ вернёт 3.

algorithm2: кандидат при проходе тоже будет 3 $\rightarrow$ вернёт 3 (и это корректно).

algorithm3: после сортировки $[1,2,3,3,3]$ блок трёх трёх идёт в конце; из-за отсутствия проверки последнего блока текущая версия может НЕ вернуть ответ (ошибка).

Пример 2 — массив без мажоритарного элемента:

$A = [1,2,3]$, $n = 3$ (никакого элемента более чем в 1.5 позиции нет)

Трассировка (кратко):

algorithm1: максимум частоты $1 \leq n/2 \rightarrow$ не возвращает элемента.

algorithm2: возвращает некоторый кандидат (например, $A[2]=3$) — неверно, если ожидать явного указания отсутствия.

algorithm3: отсортированный массив $[1,2,3]$, блоки длиной 1, условие $c > n/2$ не выполнится $\rightarrow$ не возвращает элемента (корректно).

4. Сравнение — таблица корректности и сложности

Алгоритм	Корректность без доработок	Поведение при отсутствии мажоритарного элемента	Верхняя оценка времени $O(\dots)$
algorithm1	Корректен (возвращает	Не возвращает неверный элемент;	$O(n^2)$ Внешний цикл n итераций,

	элемент только если найден $c > n/2$)	сообщает отсутствие	внутренний — $n \rightarrow n \cdot n =$ n^2 операций сравнения.
algorithm2	Находит кандидата, но без проверки — не гарантированно корректен	Может вернуть произвольный элемент (некорректно)	$O(n)$ Один проход по массиву, каждая итерация — $O(1)$.
algorithm3	После сортировки правильно находит блок, но текущая реализация не проверяет последний блок	Не возвращает при отсутствии мажоритарного элемента; может не вернуть при наличии (если последний блок)	$O(n \log n)$ Сортировка — $O(n \log n)$, проход по массиву — $O(n)$. Итог: $O(n \log n)$.

5. Формальные предложения по доработке (минимальные изменения)

Цель доработок: чтобы все алгоритмы вели себя одинаково — возвращали мажоритарный элемент, если он существует, и не возвращали ложного значения в противном случае. Изменять алгоритмы полностью нельзя, допускается только добавить/удалить инструкции.

algorithm1: не требует доработок (уже корректен).

algorithm2: добавить финальную проверку кандидата. Минимальное вставляемое тело (после основного цикла):

```
c_final = 0
for i = 0 to n-1
    if A[i] == A[ind]
        c_final = c_final + 1
if c_final > n / 2
    return A[ind]
/* иначе не возвращаем (или сообщаем об отсутствии) */
```

algorithm3: добавить проверку последнего блока после завершения цикла по отсортированному массиву. Минимальное добавление:

```
if c > n / 2
    return A[n-1]
/* или эквивалентно: проверить count(A[n-1]) > n/2 и вернуть A[n-1] */
```

6. Обоснование, что доработки не ухудшают асимптотику

algorithm2 (доработанный): основной проход $O(n)$ + один дополнительный проход для подсчёта `c_final` — ещё $O(n)$. Итого $O(n) + O(n) = O(n)$. Асимптотика не ухудшилась.

algorithm3 (доработанный): сортировка $O(n \log n)$ остаётся доминирующим; добавленная константная проверка или единичный подсчёт последнего блока — $O(1)$ или $O(n)$ в наихудшем случае при подсчёте, но оптимально можно сделать $O(1)$ проверку длины последнего блока (в переменной `c`). Итого $O(n \log n)$ остаётся.

7. Дополнительные замечания и альтернативные (эквивалентные) доработки

- Для algorithm2 вместо второго прохода можно использовать техники, уже используемые в задаче: после нахождения кандидата сразу сделать подсчёт его вхождений (одноразовый $O(n)$).
- Для algorithm3 самая надёжная и минимальная корректировка — после цикла проверить текущую длину последнего блока (значение `c`) и сделать `return A[n-1]`, если `c > n/2`.
- При желании все алгоритмы можно унифицировать: пусть каждый возвращает либо мажоритарный элемент, либо специальное значение (например, `NULL`) при его отсутствии.

8. Примеры совпадения и различий — краткое сводное перечисление

Совпадают (пример): $A = [3, 3, 3, 2, 1]$ — все три (после доработок) вернут 3.

Различаются (пример): $A = [1, 2, 3]$ — algorithm2 (без доработки) может вернуть произвольный элемент; algorithm1 и algorithm3 не вернут.

9. Заключение

Исходные алгоритмы не полностью эквивалентны. algorithm1 — корректен, но $O(n^2)$. algorithm2 — быстро находит кандидата ($O(n)$), но без проверки может возвращать неверный ответ; algorithm3 — корректный по идее ($O(n \log n)$), но в данной формулировке упускает проверку последнего блока. Добавление одной проверки в algorithm2 и одной проверки в algorithm3 делает все алгоритмы корректными и не ухудшает их асимптотику.